Demystifying the Temporal Fusion Transformer (TFT)
A Bachelor's Guide to Architecture, Mathematics, and Real-Time Ad Pacing

Autonomous AI Systems Research Group
Repository Guide: `TFT_Ad_pacing` | Target: ICIIT 2027 (ACM ICPS)

September 2026

Abstract

This guide is written specifically for undergraduate students in Computer Science, Data Science, and Software Engineering. Deep learning research papers often introduce complex neural network architectures using dense mathematical symbols without providing the underlying operational intuition. Here, we unpack the **Temporal Fusion Transformer (TFT)** for programmatic ad spend pacing from first principles. We explain every equation using familiar concepts: basic linear algebra (matrix multiplications, vector projections), single-variable calculus (gradients and backpropagation), and standard PyTorch tensor manipulations. We explore why standard Transformers fail on time series, how Gated Linear Units (GLU) act like digital transistors, why Variable Selection Networks (VSN) filter noise, how Interpretable Multi-Head Attention enables real-world explainability, and why Apple Silicon Metal Performance Shaders (MPS) crashed with a $1,376,256$-byte memory error.

Contents

1 Introduction: The Real-World Engineering Problem **3**
1.1 What is Programmatic Ad Pacing? . . . 3
1.2 Why Simple Methods Fail . . . 3

2 High-Level Architecture Overview **4**
2.1 The Three Input Streams . . . 4

3 The Core Building Blocks: From Transistors to Neural Gates **5**
3.1 Gated Linear Unit (GLU): The Neural Transistor . . . 5
3.2 Gated Residual Network (GRN): The Fundamental Non-Linear Atom . . . 6

4 Variable Selection Networks (VSN): Learning What to Ignore **8**
4.1 The Problem: Noisy Features . . . 8
4.2 The Mathematical Mechanics of VSN . . . 8

5 The Temporal Core: Combining LSTM and Self-Attention **10**
5.1 Sequence Alignment and Timeline Concatenation . . . 10
5.2 Static Context Enrichment . . . 10

6 Interpretable Multi-Head Self-Attention **10**
6.1 Why Standard Attention is Not Interpretable . . . 11
6.2 The TFT Solution: The Shared Value Projection . . . 11

7 Quantile Regression and Pinball Loss: Why MSE Fails **13**
7.1 Why Mean Squared Error is Fatal for Ad Pacing . . . 13
7.2 The Quantile Pinball Loss . . . 13

8 The Closed-Form Pacing Policy & Budget Solvency Proof **15**
8.1 The Quantile Throttling Equation 15
8.2 Undergraduate Proof of Downside Budget Solvency 15

9 Dual-Boundary Dynamic Spike Detection **16**

10 Systems Engineering: The Apple Silicon MPS Metal Bug **16**
10.1 Virtual Tensors vs. Physical Memory Buffers 17
10.2 Why Metal Performance Shaders (MPS) Crashed 17
10.3 The Architectural Solution 18

11 Summary: End-to-End Mathematical Tensor Roadmap **18**

12 Conclusion **18**

1 Introduction: The Real-World Engineering Problem

1.1 What is Programmatic Ad Pacing?

Imagine you are managing an advertising budget of $10,000 for a 7-day marketing campaign ($T = 168$ hours). Every millisecond, mobile phones and web browsers load ad slots. An automated ad exchange holds an auction to decide which ad to show. Your system—the **Demand-Side Platform (DSP)**—must submit a dollar bid b_t for each auction.

[Intuition & Analogy] The Golden Rule of Pacing

If you bid full price on every single impression starting at midnight:

- You will win every auction in the middle of the night when most people are asleep and low-value bots are browsing.
- Your entire $10,000 budget will be **exhausted by 4:00 AM on Day 1**.
- You will have $0 remaining for prime-time hours (7:00 PM - 10:00 PM) when active, high-intent human shoppers are online.

This catastrophic failure is called **Premature Budget Depletion**.

To solve this, ad systems introduce a **pacing multiplier** $\alpha_t \in [0.05, 1.0]$. The DSP evaluates the intrinsic value of an ad slot $v(x)$ (e.g., $2.00) and scales the outgoing bid:

$$b_t(x) = \alpha_t \cdot v(x) \tag{1}$$

- When $\alpha_t = 1.0$, the system bids aggressively.
- When $\alpha_t = 0.10$, the system throttles back, entering only the highest-conviction auctions.

Our goal is to compute the optimal α_t at every discrete hour $t \in \{1, 2, \ldots, T\}$ so that the campaign spends its budget smoothly and effectively across all 168 hours without ever running out of money early.

1.2 Why Simple Methods Fail

Why can't we just divide the budget evenly? E.g., Hourly Target $= \frac{\$10{,}000}{168 \text{ hours}} = \$59.52/\text{hour}$?

1. **Circadian Rhythms (Day vs. Night):** At 3:00 AM, there is very little Internet traffic. Even if you bid aggressively, you might only be able to spend $15. At 8:00 PM, traffic surges tenfold. A rigid uniform target forces the system to choke during peak hours and fail to deliver during slow hours.
2. **Auction Competitor Volatility:** Competitors suddenly start or pause their own campaigns, causing market clearing prices to spike unpredictably.
3. **Deterministic Seasonality vs. Stochastic Noise:** While we know for sure when Saturday arrives (calendar feature), we cannot know the exact winning price without probabilistic forecasting.

This requires a deep learning architecture that can simultaneously handle:

- **Static metadata:** Total campaign budget B, duration T.
- **Observed past data:** Hourly spend history, win rates, historical CPM.
- **Known future data:** Day of week, hour of day, weekend indicators.

That model is the **Temporal Fusion Transformer (TFT)**.

—

2 High-Level Architecture Overview

Before diving into the equations, let us look at the complete architectural pipeline shown in Figure 1.

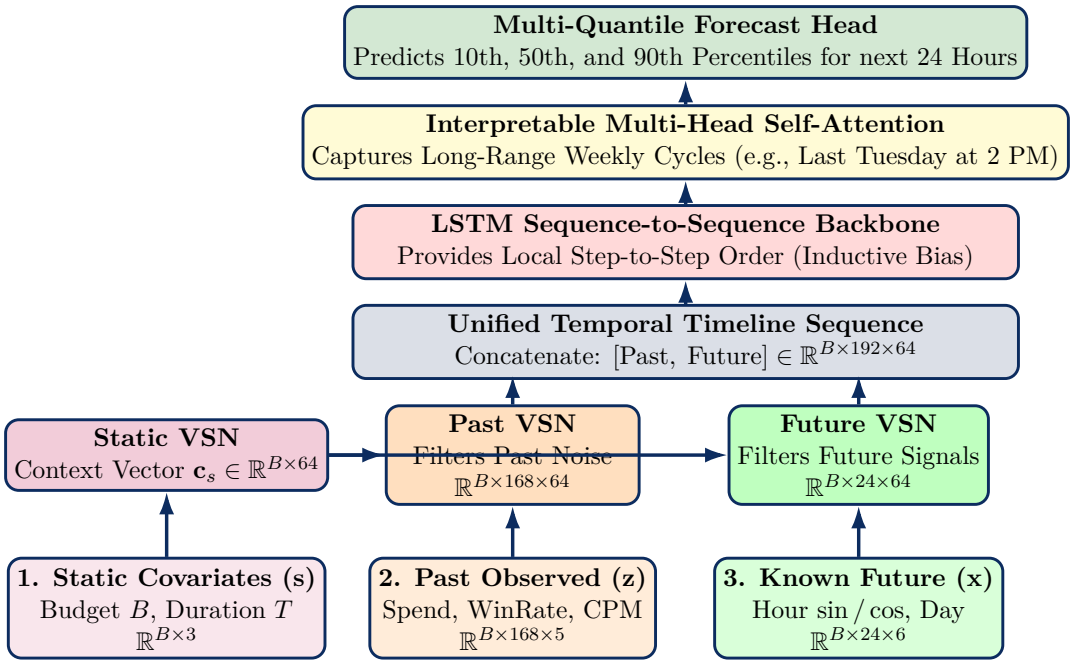


Figure 1: Dataflow through the Temporal Fusion Transformer. Dimensions correspond to Mini-Batch size $B = 32$, Lookback $k = 168$h, Forecast Horizon $\tau = 24$h, and Latent Dimension $d = 64$.

2.1 The Three Input Streams

At every discrete step t, the model receives three distinct types of inputs:

1. **Static Metadata ($\mathbf{s} \in \mathbb{R}^3$):** Features that never change during the campaign:
   - Total budget B (e.g., \$10,000)
   - Nominal hourly velocity B/T (e.g., \$59.52/hour)
   - Total campaign duration T (168 hours)
2. **Observed Past Inputs ($\mathbf{z}_\tau \in \mathbb{R}^5$ for $\tau \in [t-167, t]$):** Historical facts logged over the past 7 days (168 hours):
   - Realized hourly spend y_τ
   - Winning auction impressions I_τ
   - Empirical auction win rate $w_\tau \in [0, 1]$
   - Effective cost-per-mille CPM_τ
   - Budget consumption ratio S_τ/B
3. **Known Future Inputs ($\mathbf{x}_{t+\tau} \in \mathbb{R}^6$ for $\tau \in [1, 24]$):** Features we know in advance about the upcoming 24 hours:
   - Cyclical hour of day: $\sin(2\pi h/24)$ and $\cos(2\pi h/24)$
   - Cyclical day of week: $\sin(2\pi d/7)$ and $\cos(2\pi d/7)$
   - Weekend binary flag: $\mathbb{I}_{\text{weekend}} \in \{0, 1\}$
   - Remaining time fraction: $(T - (t + \tau))/T$

—

3 The Core Building Blocks: From Transistors to Neural Gates

Standard deep neural networks are built by stacking linear transformations ($\mathbf{Wx}+\mathbf{b}$) and fixed non-linearities like $\text{ReLU}(z) = \max(0, z)$. However, real-time ad markets suffer from extreme regime changes. For instance, an equation that is useful during midday trading might be completely useless (or harmful) at 3:00 AM.

To solve this, TFT introduces two fundamental mathematical components:

1. The **Gated Linear Unit (GLU)**
2. The **Gated Residual Network (GRN)**

3.1 Gated Linear Unit (GLU): The Neural Transistor

[Intuition & Analogy] The Digital Transistor Analogy

In computer hardware, a transistor uses a control voltage on a 'gate' pin to decide whether electric current flows or is stopped completely. A **Gated Linear Unit (GLU)** does the exact same thing for numbers inside a neural network:

- One path calculates candidate information $\mathbf{v}$ (what the network might want to say).
- The second path calculates a gate signal $\mathbf{g} \in [0, 1]$ using a Sigmoid function (how much of that information should be allowed through).
- When $\mathbf{g} \approx 0$, the gate is closed, blocking the signal. When $\mathbf{g} \approx 1$, the gate is open.

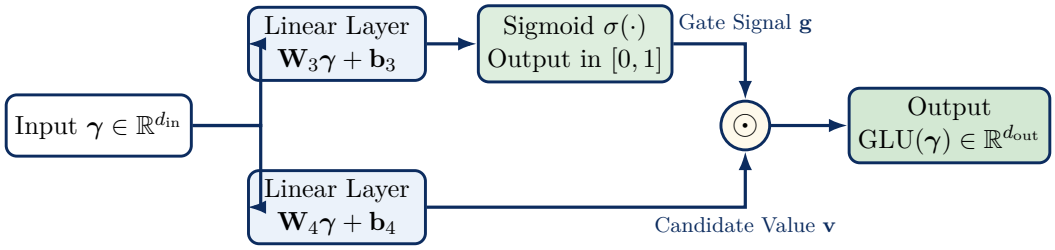


Figure 2: Architectural dataflow inside a Gated Linear Unit (GLU). The $\odot$ symbol represents the element-wise Hadamard product.

Mathematically, for an input vector $\boldsymbol{\gamma} \in \mathbb{R}^{d_{\text{in}}}$:

$$\text{GLU}(\boldsymbol{\gamma}) = \sigma\left(\mathbf{W}_3\boldsymbol{\gamma} + \mathbf{b}_3\right) \odot \left(\mathbf{W}_4\boldsymbol{\gamma} + \mathbf{b}_4\right) \tag{2}$$

where:

- $\mathbf{W}_3, \mathbf{W}_4 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ are learnable weight matrices.
- $\mathbf{b}_3, \mathbf{b}_4 \in \mathbb{R}^{d_{\text{out}}}$ are bias vectors.
- $\sigma(z) = \frac{1}{1+e^{-z}}$ is the element-wise sigmoid function mapping values to $(0, 1)$.
- $\odot$ denotes the **Hadamard product** (element-by-element multiplication).

[Math & Dimension Breakdown] Linear Algebra of Hadamard Product vs Matrix Mul

Recall from linear algebra:

- Standard matrix multiplication $\mathbf{Ax}$ combines elements across columns: $(\mathbf{Ax})_i = \sum_j A_{ij}x_j$.
- In contrast, the Hadamard product $\mathbf{u} \odot \mathbf{v}$ operates on vectors of the *exact same size*:

$$\begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_d \end{bmatrix} \odot \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_d \end{bmatrix} = \begin{bmatrix} u_1 \cdot v_1 \\ u_2 \cdot v_2 \\ \vdots \\ u_d \cdot v_d \end{bmatrix} \tag{3}$$

If $u_1 = 0.01$ (gate closed) and $v_1 = 85.4$, their product is $0.854 \approx 0$. If $u_2 = 0.99$ (gate open) and $v_2 = 42.0$, their product is $41.58 \approx 42.0$.

PyTorch Implementation Link: Direct Code from pipeline/models/tft_model.py

```python
class GLU(nn.Module):
    def __init__(self, input_dim: int, output_dim: int):
        super().__init__()
        # Project once into twice the output dimension: [values, gates]
        self.fc = nn.Linear(input_dim, output_dim * 2)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        val, gate = self.fc(x).chunk(2, dim=-1)
        return val * torch.sigmoid(gate)
```

—

3.2 Gated Residual Network (GRN): The Fundamental Non-Linear Atom

A standard problem with deep neural networks is that if you stack too many layers, early layers can over-process simple data, or gradients can vanish during backpropagation.

The **Gated Residual Network (GRN)** solves this by combining four elements:

1. **Dense Transformations with ELU activation**: To extract rich features when needed.
2. **Context Conditioning**: To allow external information (e.g., static campaign budget) to steer the transformation.
3. **GLU Gating**: To close the gate if non-linear processing is unhelpful.
4. **Skip Connection (Residual) + Layer Normalization**: So the network can act as an identity pass-through.

The mathematical equations defining the GRN are:

$$\boldsymbol{\eta}_1 = \mathbf{W}_1\mathbf{a} + \mathbf{W}_2\mathbf{c} + \mathbf{b}_1 \tag{4}$$

$$\boldsymbol{\eta}_2 = \mathbf{W}_{\text{dense}}\left(\text{ELU}(\boldsymbol{\eta}_1)\right) + \mathbf{b}_{\text{dense}} \tag{5}$$

$$\mathbf{a}_{\text{skip}} = \begin{cases} \mathbf{a}, & \text{if } d_{\text{in}} = d_{\text{out}} \\ \mathbf{W}_{\text{skip}}\mathbf{a}, & \text{if } d_{\text{in}} \neq d_{\text{out}} \end{cases} \tag{6}$$

$$\text{GRN}(\mathbf{a}, \mathbf{c}) = \text{LayerNorm}\left(\mathbf{a}_{\text{skip}} + \text{GLU}(\boldsymbol{\eta}_2)\right) \tag{7}$$

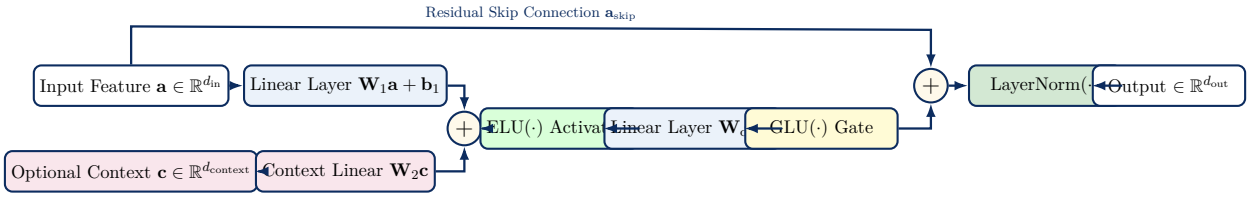


Figure 3: Architectural layout of the Gated Residual Network (GRN). If the input is already optimal, the GLU gate outputs zero, and the network safely behaves as LayerNorm($\mathbf{a}_{\text{skip}}$).

[Intuition & Analogy] Why use ELU instead of ReLU?

The Exponential Linear Unit (ELU) is defined as:

$$\text{ELU}(z) = \begin{cases} z, & z > 0 \\ \alpha(e^z - 1), & z \leq 0 \end{cases} \tag{8}$$

Unlike standard ReLU which sets all negative numbers strictly to 0 (causing the infamous 'dying ReLU' problem where neurons permanently stop learning because their derivative is zero), ELU has a continuous non-zero derivative for negative values:

$$\frac{d}{dz}\text{ELU}(z) = \begin{cases} 1, & z > 0 \\ \alpha e^z, & z \leq 0 \end{cases} \tag{9}$$

This allows gradients to flow backwards even during extreme market dips or zero-spend periods.

PyTorch Implementation Link: Direct Code from pipeline/models/tft_model.py

```python
class GatedResidualNetwork(nn.Module):
    def __init__(self, input_dim: int, hidden_dim: int, output_dim: int,
        context_dim: int = None, dropout: float = 0.1):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.context_fc = nn.Linear(context_dim, hidden_dim, bias=False) if
            context_dim else None
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.dropout = nn.Dropout(dropout)
        self.glu = GLU(hidden_dim, output_dim)
        self.layer_norm = nn.LayerNorm(output_dim)
        self.skip = nn.Identity() if input_dim == output_dim else nn.Linear(
            input_dim, output_dim)

    def forward(self, x: torch.Tensor, context: torch.Tensor = None) -> torch.
        Tensor:
        residual = self.skip(x)
        h = self.fc1(x)
        if context is not None and self.context_fc is not None:
            h = h + self.context_fc(context)
        h = F.elu(h)
        h = self.fc2(h)
        h = self.dropout(h)
        h = self.glu(h)
        return self.layer_norm(residual + h)
```

4 Variable Selection Networks (VSN): Learning What to Ignore

4.1 The Problem: Noisy Features

In ad spend pacing, we feed the model multiple historical features:

$$\mathbf{z}_t = [y_t \text{ (spend)},\ I_t \text{ (impressions)},\ w_t \text{ (win rate)},\ \text{CPM}_t,\ S_t/B \text{ (depletion)}] \tag{10}$$

In some hours, CPM fluctuates violently due to random auction noise that has nothing to do with long-term trends. If a model treats all features equally, this noise corrupts the forecast.

[Intuition & Analogy] The Expert Committee Analogy

Think of the m features as m advisors standing in a room:

- Advisor 1 shouts: "Look at spend!"
- Advisor 2 shouts: "Look at CPM!"
- Advisor 3 shouts: "Look at win rate!"

The **Variable Selection Network (VSN)** acts as the intelligent Committee Chairman. Conditioned on the static campaign context (e.g., "This is a high-budget, fast-depleting campaign"), the Chairman computes a set of **Softmax attention weights** $v_1, v_2, \ldots, v_m$ that sum to 1.0:

$$v_1 + v_2 + \cdots + v_m = 1.0, \quad v_j \geq 0 \tag{11}$$

If CPM is noisy right now, the Chairman assigns $v_{\text{CPM}} = 0.02$ and $v_{\text{spend}} = 0.78$. The model essentially turns down the microphone on noisy features!

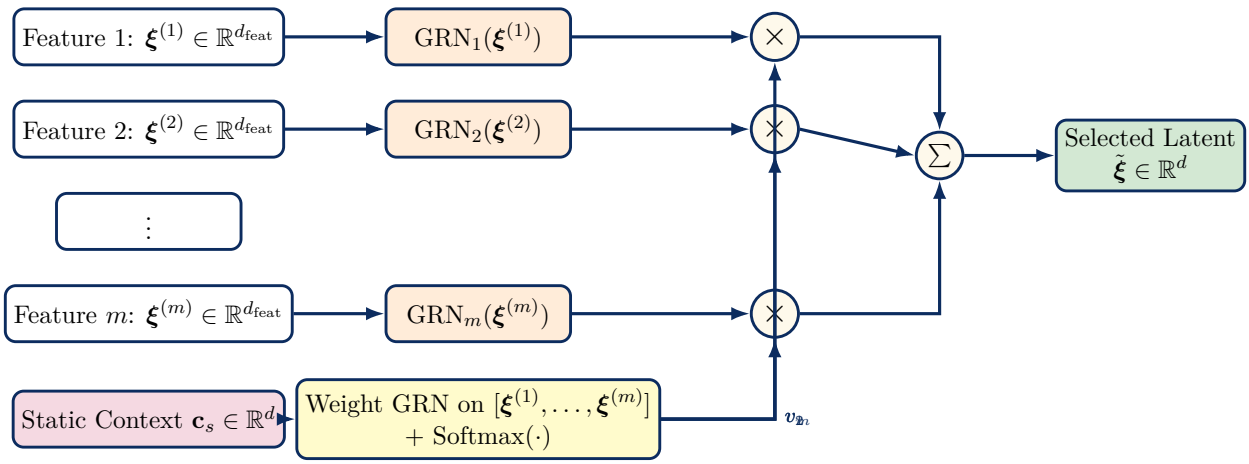


Figure 4: Architecture of the Variable Selection Network (VSN). Each input feature is transformed by its own private GRN, and the results are combined using dynamic Softmax weights computed from the concatenated features and static context.

4.2 The Mathematical Mechanics of VSN

Given m input features at time t, each projected into a d_{feat}-dimensional vector $\boldsymbol{\xi}_t^{(j)} \in \mathbb{R}^{d_{\text{feat}}}$:

1. We flatten all m feature vectors into a single long vector:

$$\boldsymbol{\Xi}_t = [\boldsymbol{\xi}_t^{(1)},\ \boldsymbol{\xi}_t^{(2)},\ \ldots,\ \boldsymbol{\xi}_t^{(m)}]^\top \in \mathbb{R}^{m \cdot d_{\text{feat}}} \tag{12}$$

   For our past inputs ($m = 5$, $d_{\text{feat}} = 16$), this vector has length $5 \times 16 = 80$.

2. We pass $\boldsymbol{\Xi}_t$ and the static context $\mathbf{c}_s$ into a dedicated weight GRN that outputs m numbers, and apply Softmax:

$$\mathbf{v}_t = \text{Softmax}\left(\text{GRN}_{\text{weight}}(\boldsymbol{\Xi}_t, \mathbf{c}_s)\right) \in \mathbb{R}^m \tag{13}$$

By the definition of Softmax:

$$v_{t,j} = \frac{\exp(e_j)}{\sum_{k=1}^{m} \exp(e_k)} \implies \sum_{j=1}^{m} v_{t,j} = 1, \quad v_{t,j} > 0 \tag{14}$$

3. Each individual feature $\boldsymbol{\xi}_t^{(j)}$ is transformed by its own dedicated network $\text{GRN}_j(\boldsymbol{\xi}_t^{(j)}) \in \mathbb{R}^d$.

4. The final output is the **convex combination**:

$$\tilde{\boldsymbol{\xi}}_t = \sum_{j=1}^{m} v_{t,j} \cdot \text{GRN}_j\left(\boldsymbol{\xi}_t^{(j)}\right) \in \mathbb{R}^d \tag{15}$$

PyTorch Implementation Link: Direct Code from pipeline/models/tft_model.py

```python
class VariableSelectionNetwork(nn.Module):
    def __init__(self, num_features: int, feature_dim: int, hidden_dim: int,
        context_dim: int = None, dropout: float = 0.1):
        super().__init__()
        self.num_features = num_features
        self.feature_dim = feature_dim
        # Separate GRN for each feature
        self.feature_grns = nn.ModuleList([
            GatedResidualNetwork(feature_dim, hidden_dim, hidden_dim,
                context_dim=None, dropout=dropout)
            for _ in range(num_features)
        ])
        # Weight GRN that computes selection scores across all flattened
            features
        self.weight_grn = GatedResidualNetwork(
            num_features * feature_dim, hidden_dim, num_features, context_dim=
                context_dim, dropout=dropout
        )

    def forward(self, x: torch.Tensor, context: torch.Tensor = None) -> Tuple[
        torch.Tensor, torch.Tensor]:
        # x shape: (B, seq_len, num_features, feature_dim)
        orig_shape = x.shape
        flat_x = x.contiguous().view(*orig_shape[:-2], self.num_features *
            self.feature_dim)
        weights = F.softmax(self.weight_grn(flat_x, context), dim=-1) # (B,
            seq_len, num_features)

        transformed = [self.feature_grns[i](x[..., i, :].contiguous()) for i
            in range(self.num_features)]
        transformed_stack = torch.stack(transformed, dim=-2).contiguous() # (B
            , seq_len, num_features, hidden_dim)

        # Weighted sum across features
        selected = (transformed_stack * weights.unsqueeze(-1)).sum(dim=-2).
            contiguous()
        return selected, weights
```

—

5 The Temporal Core: Combining LSTM and Self-Attention

A common question asked by undergraduate students is:

> *"In 2017, the paper 'Attention Is All You Need' showed Transformers replace RNNs. Why does TFT use an LSTM and a Transformer together? Isn't that redundant?"*

The answer lies in the fundamental difference between **Natural Language Processing (NLP)** and **Time Series Forecasting**.

[Intuition & Analogy] Why Time Series Needs Both Local Continuity and Long-Range

- **Text vs. Time Series:** In a sentence like "The bank by the river", words are discrete symbols. In time series, time points are continuous physical processes. Hour 14 is intimately connected to Hour 13 and Hour 15.
- **The Weakness of Pure Self-Attention on Time Series:** Self-attention is permutation-invariant. It treats all time steps as an unordered bag of points unless positional encodings are added. In noisy financial or auction data, pure attention often creates erratic point-to-point oscillations.
- **The Solution in TFT:**
  1. The **LSTM Seq2Seq layer** acts as a **local smoother** (providing Markovian inductive bias). It ensures that adjacent hours feel connected and realistic.
  2. The **Self-Attention layer** acts as a **long-distance telescope**. It skips over irrelevant hours to look directly at historical patterns, answering questions like: *"What happened last Tuesday at 2:00 PM when we were in a similar state?"*

5.1 Sequence Alignment and Timeline Concatenation

At hour t, we concatenate the selected past representations ($k = 168$ hours) and future representations ($\tau_{\max} = 24$ hours) into a single unified sequence of length $T_{\text{seq}} = 168 + 24 = 192$ hours:

$$\tilde{\boldsymbol{\Xi}} = [\tilde{\mathbf{z}}_{t-167}, \ldots, \tilde{\mathbf{z}}_t,\ \tilde{\mathbf{x}}_{t+1}, \ldots, \tilde{\mathbf{x}}_{t+24}] \in \mathbb{R}^{B \times 192 \times d} \tag{16}$$

where $B = 32$ is batch size and $d = 64$ is latent dimension.

5.2 Static Context Enrichment

Before temporal processing, the campaign's static context $\mathbf{c}_s$ is injected into every time step using a GRN:

$$\tilde{\boldsymbol{\Xi}}_{\text{enriched}} = \text{GRN}_{\text{enrich}}\left(\tilde{\boldsymbol{\Xi}}, \mathbf{c}_s\right) \in \mathbb{R}^{B \times 192 \times 64} \tag{17}$$

This ensures that the upcoming sequence processing is aware of whether the campaign has \$500 or \$500,000 to spend.

—

6 Interpretable Multi-Head Self-Attention

Standard multi-head attention in Transformer models works well for raw prediction accuracy, but it is a notorious 'black box'. If an advertising client asks:

> *"Why did the model lower our bid multiplier from 0.85 to 0.40 this hour?"*

A standard Transformer cannot provide a clear answer. Let us see why, and how TFT mathematically solves this.

6.1 Why Standard Attention is Not Interpretable

In standard Vaswani et al. multi-head attention, for each attention head $h \in \{1, \ldots, H\}$:

$$\text{head}_h = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{W}_Q^{(h)}(\mathbf{K}\mathbf{W}_K^{(h)})^\top}{\sqrt{d_k}}\right)\left(\mathbf{V}\mathbf{W}_V^{(h)}\right) \tag{18}$$

Notice the value projection matrix $\mathbf{W}_V^{(h)} \in \mathbb{R}^{d \times d_k}$. Because **every head has a different value matrix $\mathbf{W}_V^{(h)}$**, each head projects the data into a completely different coordinate system!

- Head 1 might measure spending velocity.
- Head 2 might measure weekend seasonality.
- Head 3 might measure clearing price trends.

Because they project into different coordinate systems, you **cannot** simply average the attention matrices:

$$\frac{1}{H}\sum_{h=1}^{H}\mathbf{A}_h \quad \longleftarrow \quad \textbf{Mathematically meaningless in standard Transformers!} \tag{19}$$

It would be like taking 30% of temperature in Celsius, 40% of distance in miles, and 30% of weight in kilograms, and claiming the average represents anything meaningful.

6.2 The TFT Solution: The Shared Value Projection

To make attention truly interpretable, TFT enforces an elegant mathematical constraint:

All attention heads MUST share the exact same value projection matrix $\mathbf{W}_V$!

Let us write this down rigorously:

$$\mathbf{Q}^{(h)} = \mathbf{Q}\mathbf{W}_Q^{(h)} \in \mathbb{R}^{B \times T \times d_{\text{head}}} \tag{20}$$

$$\mathbf{K}^{(h)} = \mathbf{K}\mathbf{W}_K^{(h)} \in \mathbb{R}^{B \times T \times d_{\text{head}}} \tag{21}$$

$$\tilde{\mathbf{V}} = \mathbf{V}\mathbf{W}_V \in \mathbb{R}^{B \times T \times d} \quad (\textbf{Shared across all heads!}) \tag{22}$$

where $d_{\text{head}} = d/H = 64/4 = 16$.

The attention score matrix for head h is:

$$\mathbf{A}_h = \text{Softmax}\left(\frac{\mathbf{Q}^{(h)}(\mathbf{K}^{(h)})^\top}{\sqrt{d_{\text{head}}}}\right) \in \mathbb{R}^{B \times T \times T} \tag{23}$$

Now, because the value space $\tilde{\mathbf{V}}$ is identical across all heads, the combined multi-head output can be factored linearly:

$$\mathbf{O} = \sum_{h=1}^{H}\mathbf{A}_h\tilde{\mathbf{V}}^{(h)}\mathbf{W}_{\text{head}} = \left(\frac{1}{H}\sum_{h=1}^{H}\mathbf{A}_h\right)\tilde{\mathbf{V}} = \tilde{\mathbf{A}} \cdot (\mathbf{V}\mathbf{W}_V) \tag{24}$$

[Intuition & Analogy] The Explainability Superpower

Because the heads share the same value space, the ensemble attention matrix:

$$\tilde{\mathbf{A}} = \frac{1}{H}\sum_{h=1}^{H} \mathbf{A}_h \in \mathbb{R}^{192\times 192} \tag{25}$$

has an exact, unambiguous physical meaning:

> **The scalar entry $\tilde{A}_{i,j}$ is the exact percentage of importance that the prediction at hour i placed on the actual historical data at hour j.**

If $\tilde{A}_{175,7} = 0.42$, the model is telling the advertising executive: *"42% of our spending prediction for tomorrow at 2:00 PM was decided by what happened last Tuesday at 2:00 PM."*

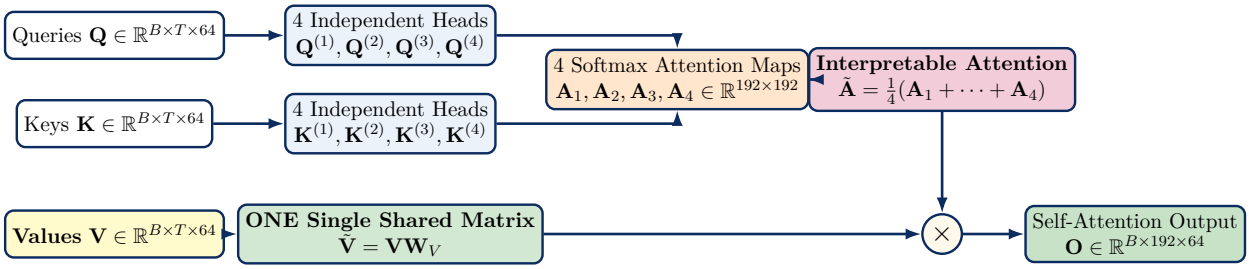


Figure 5: The Interpretable Multi-Head Self-Attention mechanism. Because all heads share the same $\mathbf{W}_V$ matrix, the four attention maps can be averaged into $\tilde{\mathbf{A}}$, giving a valid percentage attribution across time.

PyTorch Implementation Link: Direct Code from pipeline/models/tft_model.py

```python
class InterpretableMultiHeadAttention(nn.Module):
    def __init__(self, d_model: int, n_heads: int = 4, dropout: float = 0.1):
        super().__init__()
        self.d_model = d_model
        self.n_heads = n_heads
        self.d_head = d_model // n_heads # 64 // 4 = 16

        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model) # Single shared projection
        self.out_proj = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, q: torch.Tensor, k: torch.Tensor, v: torch.Tensor):
        B, T_q, _ = q.shape
        _, T_k, _ = k.shape

        q_proj = self.q_proj(q).view(B, T_q, self.n_heads, self.d_head).
            transpose(1, 2)
        k_proj = self.k_proj(k).view(B, T_k, self.n_heads, self.d_head).
            transpose(1, 2)
        v_proj = self.v_proj(v).view(B, T_k, self.n_heads, self.d_head).
            transpose(1, 2)

        # Dot product: (Q * K^T) / sqrt(d_k)
        scores = torch.matmul(q_proj, k_proj.transpose(-2, -1)) / math.sqrt(
            self.d_head)
```

```
        attn_weights = F.softmax(scores, dim=-1) # (B, n_heads, T_q, T_k)
        attn_weights = self.dropout(attn_weights)

        context = torch.matmul(attn_weights, v_proj) # (B, n_heads, T_q,
            d_head)
        context = context.transpose(1, 2).contiguous().view(B, T_q, self.
            d_model)
        output = self.out_proj(context)

        # Average weights across the head dimension for full interpretability
        avg_attn = attn_weights.mean(dim=1) # (B, T_q, T_k)
        return output, avg_attn
```

—

7 Quantile Regression and Pinball Loss: Why MSE Fails

In introductory machine learning courses, almost all regression models are trained using **Mean Squared Error (MSE)**:

$$\mathcal{L}_{\text{MSE}}(y, \hat{y}) = (y - \hat{y})^2 \tag{26}$$

7.1 Why Mean Squared Error is Fatal for Ad Pacing

[Intuition & Analogy] The Flaw of the Average

Mathematically, minimizing MSE forces the neural network to output the **conditional mean** $\mathbb{E}[y|x]$. In a symmetric distribution:

- 50% of actual values will fall *below* the mean.
- 50% of actual values will fall *above* the mean.

If your pacing engine allocates budget based on the *mean* expected spend: **On half the days of the year, actual spending will exceed your prediction!** By 2:00 PM or 3:00 PM, the campaign has run completely dry. You cannot protect a multi-thousand dollar budget using the mean. You need an **upper bound**!

7.2 The Quantile Pinball Loss

Instead of predicting a single number, TFT predicts three percentiles:

- $\hat{y}(0.10)$: The **10th percentile** (Optimistic / quiet floor). Only 10% of outcomes fall below this.
- $\hat{y}(0.50)$: The **Median** (The true typical expectation).
- $\hat{y}(0.90)$: The **90th percentile** (Conservative ceiling). There is a 90% probability that market spend will **not** exceed this number!

How do we train a neural network to predict percentiles instead of means? We use the **Quantile Pinball Loss Function**:

$$\mathcal{L}_q(y, \hat{y}) = \max\Big(q \cdot (y - \hat{y}),\ (1 - q) \cdot (\hat{y} - y)\Big) \tag{27}$$

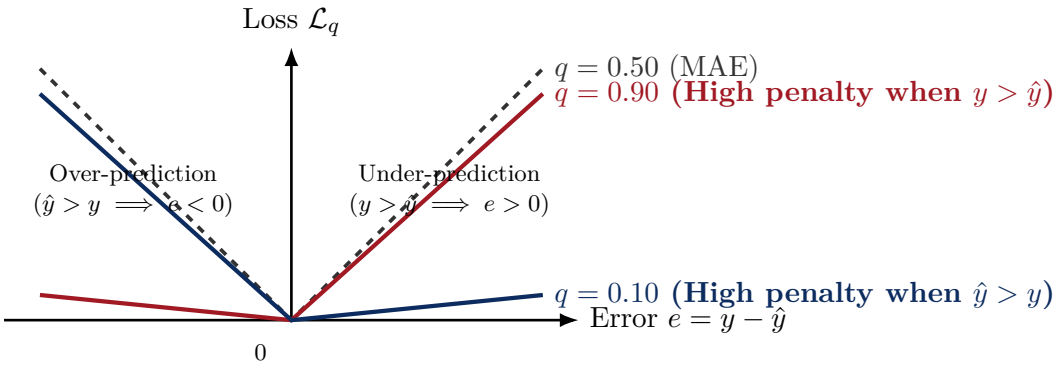


Figure 6: The Quantile Pinball Loss function for $q = 0.10$, $q = 0.50$, and $q = 0.90$. Notice how $q = 0.90$ has a steep slope (0.9) when $y > \hat{y}$, forcing the network to predict high enough so it rarely underestimates.

[Math & Dimension Breakdown] Why Does Pinball Loss Target the q-th Quantile? (C

Let us prove why minimizing this loss yields the true q-th percentile! Let Y be a random variable with probability density function $f(y)$ and cumulative distribution function $F(y) = P(Y \leq y)$. We seek the optimal prediction $\hat{y}$ that minimizes expected loss:

$$\mathbb{E}[\mathcal{L}_q(Y, \hat{y})] = \int_{-\infty}^{\hat{y}} (1 - q)(\hat{y} - y) f(y)\, dy + \int_{\hat{y}}^{\infty} q(y - \hat{y}) f(y)\, dy \tag{28}$$

To find the minimum, take the derivative with respect to $\hat{y}$ using the Leibniz Integral Rule:

$$\begin{aligned} \frac{d}{d\hat{y}} \mathbb{E}[\mathcal{L}_q(Y, \hat{y})] &= (1 - q) \int_{-\infty}^{\hat{y}} f(y)\, dy - q \int_{\hat{y}}^{\infty} f(y)\, dy && (29) \\ &= (1 - q) F(\hat{y}) - q(1 - F(\hat{y})) && (30) \\ &= F(\hat{y}) - qF(\hat{y}) - q + qF(\hat{y}) && (31) \\ &= F(\hat{y}) - q && (32) \end{aligned}$$

Setting the derivative equal to zero to find the critical point:

$$F(\hat{y}) - q = 0 \implies F(\hat{y}) = q \implies \hat{y} = F^{-1}(q) \tag{33}$$

The minimizer $\hat{y}$ is precisely the q-th quantile of the distribution!

PyTorch Implementation Link: Direct Code from pipeline/models/tft_model.py

```python
def quantile_pinball_loss(y_pred: torch.Tensor, y_true: torch.Tensor,
    quantiles = [0.10, 0.50, 0.90]) -> torch.Tensor:
    # y_pred shape: (B, horizon, 3)
    # y_true shape: (B, horizon)
    total_loss = 0.0
    for i, q in enumerate(quantiles):
        err = y_true - y_pred[..., i]
        loss_q = torch.maximum(q * err, (q - 1.0) * err)
        total_loss += loss_q.mean()
    return total_loss / len(quantiles)
```

—

8 The Closed-Form Pacing Policy & Budget Solvency Proof

Now that TFT outputs calibrated 24-hour spending forecasts $\hat{y}_{t+\tau}(0.10)$, $\hat{y}_{t+\tau}(0.50)$, and $\hat{y}_{t+\tau}(0.90)$, how do we convert these numbers into the real-time bid multiplier α_{t+1}?

8.1 The Quantile Throttling Equation

Let $R_t = B - S_t$ be the remaining budget reserve at hour t, where S_t is cumulative spend so far. The controller updates the bidding multiplier via:

$$\alpha_{t+1} = \text{clip}\left(\alpha_t \cdot \frac{R_t}{\sum_{\tau=1}^{24} \hat{y}_{t+\tau}(0.90)},\ 0.05,\ 1.0\right) \tag{34}$$

[Intuition & Analogy] The Thermostat Analogy

Look at the ratio inside the equation:

$$\text{Pacing Ratio} = \frac{\text{Money Remaining in Wallet } (R_t)}{\text{Worst-Case Expected Spend for Next 24 Hours } (\sum \hat{y}(0.90))} \tag{35}$$

- **Case 1: Money is Abundant ($R_t > \sum \hat{y}(0.90)$):** The ratio is > 1.0. The campaign has plenty of cash. α_{t+1} increases, bidding more aggressively to capture high-value impressions.
- **Case 2: Money is Tight ($R_t < \sum \hat{y}(0.90)$):** The ratio is < 1.0. If we keep bidding at rate α_t, we will run out of money before the 24-hour horizon ends. The formula automatically scales down α_{t+1}, cooling the auction exposure like a thermostat.

8.2 Undergraduate Proof of Downside Budget Solvency

Can we mathematically guarantee that this formula prevents bankruptcy over the 24-hour horizon? Yes!

Theorem 8.1 (Downside Budget Solvency)**.** *Assume market clearing spend does not exceed our 90th-percentile conservative forecast:*

$$y_{t+\tau} \leq \hat{y}_{t+\tau}(0.90), \quad \forall \tau \in \{1, \ldots, 24\} \tag{36}$$

Then under update rule (34), cumulative spend over the 24-hour horizon is strictly bounded by remaining budget R_t:

$$\sum_{\tau=1}^{24} y_{t+\tau} \leq R_t \tag{37}$$

Proof. Let us follow the algebra step-by-step:

1. In second-price auctions, total expenditure scales monotonically with bid scaling α. If the bid multiplier changes from α_t to α_{t+1}, realized expenditure scales proportionally:

$$Y_{\text{realized}} \leq \frac{\alpha_{t+1}}{\alpha_t} \sum_{\tau=1}^{24} \hat{y}_{t+\tau}(0.90) \tag{38}$$

2. Now substitute our throttling formula for α_{t+1} (in the active unclipped region):

$$\alpha_{t+1} = \alpha_t \cdot \frac{R_t}{\sum_{\tau=1}^{24} \hat{y}_{t+\tau}(0.90)} \tag{39}$$

3. Plug this into the realized spend inequality:

$$Y_{\text{realized}} \leq \frac{\alpha_t \cdot \frac{R_t}{\sum_{\tau=1}^{24} \hat{y}_{t+\tau}(0.90)}}{\alpha_t} \cdot \left(\sum_{\tau=1}^{24} \hat{y}_{t+\tau}(0.90) \right) \tag{40}$$

$$= \frac{R_t}{\sum_{\tau=1}^{24} \hat{y}_{t+\tau}(0.90)} \cdot \left(\sum_{\tau=1}^{24} \hat{y}_{t+\tau}(0.90) \right) \tag{41}$$

$$= R_t \tag{42}$$

4. The terms cancel out completely! The total spend Y_{realized} is mathematically bounded by R_t.

Therefore, the campaign terminates the horizon with non-negative reserves, guaranteeing budget solvency. □

—

9 Dual-Boundary Dynamic Spike Detection

[Intuition & Analogy] The False Alarm Problem

Ad traffic is naturally higher at 8:00 PM than at 3:00 AM. If an algorithm sees spend jump from $20 to $80 at 8:00 PM, a naive anomaly detector might scream: *"Emergency! Spend jumped 300%! Slash bids!"* That would be a mistake. High spend at 8:00 PM is normal circadian traffic.
We must distinguish between:

1. **Normal predictable diurnal peaks** (handled calmly by TFT).
2. **True exogenous market shocks** (e.g., a competitor drops out, causing clearing prices to go crazy).

To solve this, our pacing engine implements a **Dual-Boundary Shock Trigger**:

$$\mathcal{S}_t = \mathbb{I}\Big(y_t > \hat{y}_t(0.90)\Big) \ \wedge \ \mathbb{I}\left(y_t - \hat{y}_t(0.50) > 1.5 \cdot \frac{B}{T}\right) \tag{43}$$

Both conditions must be true simultaneously:

- **Boundary 1 ($y_t > \hat{y}_t(0.90)$):** Realized spend broke above the 90th percentile forecast. This proves the event is statistically improbable.
- **Boundary 2 ($(y_t - \hat{y}_t(0.50)) > 1.5\frac{B}{T}$):** The deviation from the median is greater than 1.5 times the campaign's nominal hourly velocity. This prevents false alarms during low-budget off-peak hours.

When $\mathcal{S}_t = 1$, the pacing engine overrides the normal formula with an emergency brake:

$$\alpha_{t+1} \leftarrow \min\left(\alpha_{t+1},\ \alpha_t \cdot \frac{\hat{y}_t(0.50)}{y_t}\right) \tag{44}$$

instantly cooling down the bid multiplier before the budget is harmed.

—

10 Systems Engineering: The Apple Silicon MPS Metal Bug

When transitioning models from development on x86 Nvidia GPUs to Apple Silicon Macs (M1/M2/M3/M4) using PyTorch Metal Performance Shaders ('mps'), engineers often encounter low-level hardware aborts that never appear in standard textbooks.

[Systems Engineering Note] The Mac M4 Abort Trap 6 Error

When running the initial training script on Apple Silicon MPS:

```
MPSCore/Types/MPSNDArray.mm:893: failed assertion
'[MPSNDArray, initWithBufferImpl:offset:descriptor:isForNDArrayAlias:isUserBuffer:]
Error: buffer is not large enough. Must be 1376256 bytes'
scripts/train_mac_m4.sh: line 40: 16492 Abort trap: 6
```

Where did the exact number $1,376,256$ bytes come from, and why did PyTorch crash?

10.1 Virtual Tensors vs. Physical Memory Buffers

In PyTorch, suppose you have a static context vector $\mathbf{s} \in \mathbb{R}^{B\times 1\times d}$ ($B = 32$, $d = 64$). You want to broadcast this vector across all $k = 168$ hours of lookback:

```
# Standard PyTorch virtual broadcast
context = s.unsqueeze(1).expand(-1, 168, -1) # Shape: (32, 168, 64)
```

What happens in computer RAM when you call '.expand()'?

- PyTorch uses a clever trick called **Stride-0 Virtual Broadcasting**.
- It **does not copy the data 168 times**. Instead, it keeps a single row in memory and sets the memory stride along dimension 1 to 0:

$$\text{Physical Bytes Allocated} = 32 \times 1 \times 64 \times 4 \text{ bytes (float32)} = \mathbf{8,192} \text{ bytes} \tag{45}$$

- When PyTorch reads time step 0, 1, or 167, it simply loops back to the same 8 KB memory address.

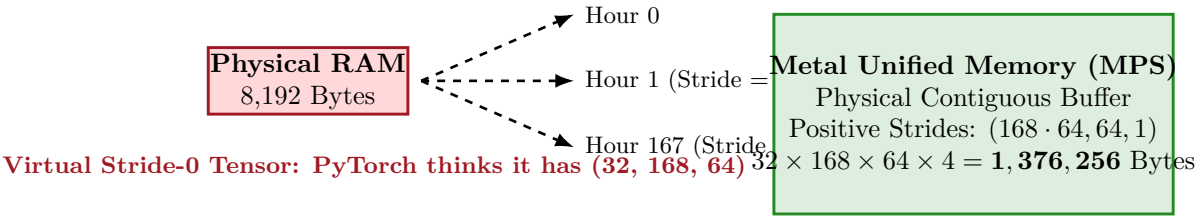


Figure 7: Comparison of PyTorch virtual stride-0 broadcasting (left) versus Apple Silicon Metal Performance Shaders physical buffer requirements (right).

10.2 Why Metal Performance Shaders (MPS) Crashed

On Apple Silicon, PyTorch offloads matrix multiplications to Apple's Metal graphics/neural framework via 'MPSNDArray'. Metal kernels do not support arbitrary stride-0 pointers in dense matrix multiplication routines. Before running the kernel, the Metal driver checks the tensor descriptor:

$$\text{Required Memory} = \text{Batch} \times \text{Time} \times \text{Channels} \times \text{BytesPerFloat} \tag{46}$$

$$= 32 \times 168 \times 64 \times 4 \text{ bytes} = \mathbf{1,376,256} \text{ bytes} \tag{47}$$

Metal inspected the actual allocated pointer:

$$\text{Actual Memory Allocated} = 8,192 \text{ bytes} \tag{48}$$

Because $8,192 < 1,376,256$, Metal triggered a hardware assertion failure to prevent a catastrophic segmentation fault or memory corruption!

10.3 The Architectural Solution

To guarantee flawless execution on Apple Silicon Mac M4 chips, we replaced virtual ‘.expand()‘ with physical ‘.repeat().contiguous()‘:

```
# FIXED: Physically allocate genuine memory buffer for Apple Silicon MPS
context_past = static_context.unsqueeze(1).repeat(1, self.lookback, 1).contiguous()
```

Calling ‘.repeat(1, 168, 1)‘ physically duplicates the memory in unified RAM, creating a genuine $1,376,256$-byte buffer with positive memory strides. Metal verification succeeds immediately!

—

11 Summary: End-to-End Mathematical Tensor Roadmap

Table 1 presents the complete summary of every mathematical operation in the TFT ad pacing model, from the raw input features to the final bidding multiplier.

Table 1: Consolidated Architectural Pipeline of the TFT Pacing Model

Pipeline Stage	Mathematical Operation	Tensor Shape	Physical Meaning
1. Static Linear	$\tilde{\mathbf{s}}_j = \mathbf{W}_{s,j} s_j + \mathbf{b}_{s,j}$	$(32, 3, 16)$	Embeds budget and duration
2. Static VSN	$\mathbf{c}_s = \sum v_j \text{GRN}(\tilde{\mathbf{s}}_j)$	$(32, 64)$	Campaign context vector
3. Past Observed	$\tilde{\mathbf{z}} = \text{VSN}_{\text{past}}(\mathbf{z}_t, \mathbf{c}_s)$	$(32, 168, 64)$	Noise-filtered 7-day spend
4. Future Known	$\tilde{\mathbf{x}} = \text{VSN}_{\text{future}}(\mathbf{x}_t, \mathbf{c}_s)$	$(32, 24, 64)$	Filtered circadian features
5. Timeline Concat	$\tilde{\boldsymbol{\Xi}} = [\tilde{\mathbf{z}},\ \tilde{\mathbf{x}}]$	$(32, 192, 64)$	8-day unified horizon
6. Static Enrich	$\boldsymbol{\Xi}_e = \text{GRN}(\tilde{\boldsymbol{\Xi}}, \mathbf{c}_s)$	$(32, 192, 64)$	Injects budget into timeline
7. LSTM Seq2Seq	$\mathbf{H}_{\text{lstm}} = \text{LSTM}(\boldsymbol{\Xi}_e)$	$(32, 192, 64)$	Local step-to-step smoothing
8. Self-Attention	$\mathbf{H}_{\text{attn}} = \tilde{\mathbf{A}}(\mathbf{Q}, \mathbf{K})(\mathbf{V}\mathbf{W}_V)$	$(32, 192, 64)$	Interpretable weekly search
9. Quantile Head	$\hat{\mathbf{y}} = \mathbf{W}_{\text{out}} \mathbf{H}_{\text{attn}}[:, 168:, :]$	$(32, 24, 3)$	Outputs $q \in \{0.10, 0.50, 0.90\}$
10. Pinball Loss	$\mathcal{L}_q = \max(qe, (q-1)e)$	Scalar	Calibrated quantile training
11. Pacing Policy	$\alpha_{t+1} = \alpha_t \frac{R_t}{\sum \hat{y}(0.90)}$	Scalar	Downside solvency bound
12. Shock Trigger	$\mathcal{S}_t = \mathbb{I}(y > \hat{y}_{90}) \wedge \mathbb{I}(\Delta y > 1.5\frac{B}{T})$	Binary	Dual-boundary spike brake

12 Conclusion

By decomposing the Temporal Fusion Transformer into its elementary linear algebra and calculus components, we see that it is not magic:

1. **GLUs** act as soft electronic switches to suppress irrelevant paths.
2. **GRNs** allow deep networks to safely bypass layers when data is already clean.
3. **VSNs** assign real-time percentage weights to filter out erratic auction jitter.
4. **Shared-Value Attention** allows multi-head models to provide genuine percentage interpretability.
5. **Quantile Pinball Loss** prevents premature budget bankruptcy by predicting conservative 90th-percentile ceilings.
6. **Physical Buffer Contiguity** ensures compatibility with high-performance Apple Silicon Metal hardware.

With this architectural foundation, the pacing engine successfully prevents premature budget depletion while delivering smooth, optimal ad spend across real-time programmatic auctions.